

Tutorial 2.1 - Epsilon-Greedy and Calculating Regret (Solutions)

LfBD Team, 2026

September 2026

Introduction

In this problem set, you will be implementing two concepts.

- **The ϵ -Greedy Algorithm:** In short, this algorithm picks in ϵ % of cases a random arm (exploration). In $1-\epsilon$ % of cases, pick the arm with at that point the highest average reward (exploitation).
- **Regret calculation:** in this case, regret means the reward the algorithm did not obtain because it did not play the optimal arm. This means the difference between the rewards obtained by pulling the chosen arms (in this case, the arms selected by the ϵ -Greedy algorithm) and the rewards that would have been obtained had we pulled the optimal arms.

In addition to the concepts mentioned above, we will use more pipe operations and apply data subsetting. If you'd like additional material and practice with these techniques before continuing the tutorial, extra exercises are available in the GitHub repository. You can find them in the markdown file 'SP_questions,' with the corresponding answers in 'SP_questions_ANSWERS'.

Loading libraries

Before starting the problem set, make sure you have all the libraries installed that are needed. Simply run this chunk below.

```
# Packages required for subsequent analysis. P_load ensures these will be installed and loaded.
if (!require("pacman")) install.packages("pacman")
pacman::p_load(dplyr,
               tidy,
               ggplot2,
               reshape2,
               latex2exp
               )

# Additionally we load some custom functions to help us later on
source("helper_functions.R")

# If you do not have tinytex installed, uncomment the command below for knitting the pdf
#tinytex::install_tinytex()
```

Dataset

We will explore the concepts from this problem set with a dataset from Yahoo!. This dataset contains information on 10 articles, detailing when each article was shown to a user and whether the user clicked on it. The data is from a single day and contains +40k observations. If your laptop cannot handle the size of this

dataset, we have provided a smaller version with fewer observations for you to use. Please note that results may vary depending on which dataset you use. The dataset contains three columns:

- **arm:** the index of the article shown to the user (1-10)
- **reward:** 0 if a user did not click on the article, 1 if a user did click on the article
- **index:** unique id of the arm-reward combination.

Each observation is an arm and the respective reward for showing that arm to a user. The articles were randomly shown to users, which makes this dataset suitable for evaluating our ϵ -Greedy Algorithm.

```
# reads in full csv of Yahoo dataset
dfYahoo <- read.csv('yahoo_day1_10arms_tiny.csv')[,-c(1,2)]

# selects the two relevant columns from Yahoo dataset; arm shown to user and reward observed
dfYahoo_for_sim <- dfYahoo %>%
  select(arm, reward)

# Create an index column
dfYahoo_for_sim$index <- 1:nrow(dfYahoo_for_sim)
```

ϵ -Greedy Algorithm: notation

First, a brief recap of key notation from the lecture that will be useful for this tutorial.

With a denoting an arm, $a \in \mathcal{A}$ indicates that the arm a is one of the options available to our algorithm from the set of all available arms, $\mathcal{A}$. We define a_t as the chosen arm at time t . Our goal is to select arms in a way that gives us the highest possible reward, $Q_t(a)$. In the case of an ϵ -Greedy Algorithm, we pursue the greedy option in $(1 - \epsilon)\%$ of cases, and in $\epsilon\%$ of cases we select a random arm. Thus, ϵ determines the degree of exploitation (greedy) vs. exploration (random choice). More formally, in the case of an ϵ -Greedy Algorithm, our chosen arm becomes:

$$a_t = \begin{cases} \operatorname{argmax}_{a \in \mathcal{A}} Q_t(a) & \text{with prob. } 1 - \epsilon \quad (\text{exploitation}) \\ a \in_R \mathcal{A} & \text{with prob. } \epsilon \quad (\text{exploration}) \end{cases} \quad (1)$$

where $a \in_R \mathcal{A}$ denotes a randomly chosen arm from the set of available arms.

ϵ -Greedy Algorithm: code

We will code up our own version of the ϵ -Greedy algorithm in two steps. First, you will code a function that is based on a set of arms and rewards, selects an arm according to the ϵ -Greedy algorithm. Second, this tutorial will provide a function for simulating the ϵ -Greedy algorithm on the Yahoo dataset.

Task 1: finish the function specified below

Given a set of arms and rewards, we want to write a function, to be called ‘policy_greedy’, that selects an arm. It should do so in line with equation (1).

IMPORTANT: The outline of the function is specified below - you only have to fill in the part that states ‘TODO’.

Your function should return an integer which indexes which arm (1-10) is the chosen arm. In order to check if your code works, set $\epsilon = 0$ - it then should return 3. Then, set $\epsilon = 0.5$. Now, in 50% of cases it should return 3, and in 50% cases another random number.

```

# Grab the first 100 observations, create dataframe to practice for policy_greedy() function
df_practice <- dfYahoo_for_sim[1:100,]

# set value of epsilon parameter
eps <- 0

#####
# policy_greedy : This function picks an arm, based on the greedy algorithm for multi-armed bandits
#
# The policy_greedy function takes the following three arguments as input :
# df: this is a data.frame with three columns: arm, reward, and index
# eps : a float, which is the percentage of times a random arm needs to be picked
# n_arms: an integer, which is the total number of arms available. Default value is set to 10.
#
# The function outputs/returns :
# chosen_arm ; integer, index of the arm chosen
#####
policy_greedy <- function(df, eps, n_arms=10){

  # Draws a random float between 0 and 1 from a uniform distribution
  random_uniform_variable <- runif(1, min=0, max=1)

  # in epsilon % of cases, pick a random arm (exploration)
  if (random_uniform_variable < eps){

    # TODO: sample uniform random arm between 1:n_arms
    chosen_arm <- sample(1:n_arms,1)

  }

  # in (1-epsilon)% of cases, pick the arm that has the highest average reward
  else{

    # TODO: finish the code below to create the dataframe df_reward_overview
    # this dataframe shows for each arm:
    # - reward_size: total reward
    # - sample_size: total observations
    # - succes_rate: total reward/total observations

    df_reward_overview <- df %>%
      drop_na() %>%
      group_by(arm) %>%
      summarise(reward_size = sum(reward, na.rm=TRUE),
                sample_size = n(),
                succes_rate=reward_size/sample_size)

    # TODO: pick the arm with the highest success_rate from df_reward_overview
    chosen_arm <- which.max(df_reward_overview$succes_rate)

  }

  # return the chosen arm
  return(chosen_arm)
}

```

```
}
```

Now that we have this function, we can use it to simulate an ϵ -Greedy algorithm for the Yahoo dataset. Before the simulation starts, there is an initial period in which we gather some information about the arms. In this period, users are randomly exposed to articles. Based on this initial information, we then start our ϵ -Greedy algorithm to pick the articles. For $n_{\text{before simulation}}$ steps we gather information, and for $n_{\text{simulation}}$ steps we simulate our algorithm.

The function on the next page simulates the performance of our ϵ -Greedy algorithm.

Task 2: finish the function specified below

```
###
# sim_greedy: simulates performance of an epsilon-greedy policy
#
# Arguments:
#
#   df: data.frame with column names "arm", "reward", and "index"
#
#   n_before_sim: integer, number of observations (randomly sampled)
#                 before starting the epsilon-greedy algorithm
#
#   n_sim: integer, number of observations used to simulate the
#          performance of the epsilon-greedy algorithm
#
#   epsilon: float between 0 and 1. In epsilon% of cases,
#            randomly explore other arms. In 1-epsilon %, pick arm
#            that has highest average reward up until that point.
#
#   interval: the number of steps after which our arm is updated.
#             For example, interval is 5 means that when an arm
#             is chosen by our approach, it is deployed for 5 steps.
#             The default value is set to 1.
#
#
# Output: list with following
#   df_results_of_policy: n_sim x 2 data.frame, with "arm", "reward".
#                       Random sampled rewards for chosen arms
#
#   df_sample_of_policy: data.frame with sample used to evaluate policy.
#
#
###
sim_greedy <- function(df, n_before_sim, n_sim, epsilon, interval=1){

  ## Part 1: create two dataframes, one with data before start of policy, and one with data after

  # define the number of observations of all data available
  n_obs <- nrow(df)

  # Give user a warning: the size of the intended experiment is bigger than the data provided
  if(n_sim > (n_obs-n_before_sim)){
    stop("The indicated size of the experiment is bigger than the data provided - shrink the size ")
  }
}
```

```

# Next we prepare our dataframes using functions from helper_functions.R
prepped_data <- prepare_dataframes(df, n_obs, n_before_sim, n_sim)

# Access individual data frames (Explain these more)
df_results_policy <- prepped_data$df_results_policy
df_during_policy <- prepped_data$df_during_policy
df_results_at_t <- prepped_data$df_results_at_t

## part 2: apply epsilon-greedy algorithm, updating at interval
for (i in 1:n_sim){

  # update at interval
  if((i==1) || ((i %% interval)==0)){

    # TODO: select the arm with your policy_greedy function and define the current arm
    chosen_arm <- policy_greedy(df_results_at_t, epsilon)
    current_arm <- chosen_arm

  }
  else{

    # TODO: In the case that we do not update, take the current arm
    chosen_arm <- current_arm
  }

  # select from the data for experiment the arm chosen
  df_during_policy_arm <- df_during_policy %>%
    filter(arm==chosen_arm)

  # warn the user to increase the dataset or downsize the experiment,
  # in the case that we have sampled all observations from an arm
  if(nrow(df_during_policy_arm) == 0){
    stop("You have run out of observations from a chosen arm")
  }

  # randomly sample from this arm and observe the reward
  sampled_arm <- sample(1:nrow(df_during_policy_arm), 1)
  reward <- df_during_policy_arm$reward[sampled_arm ]

  # important: remove the reward from the dataset to prevent repeated sampling
  index_result <- df_during_policy_arm$index[sampled_arm]
  df_during_policy <- df_during_policy %>% filter(index != index_result)

  # Get a vector of results from the chosen arm (arm, reward)
  result_policy_i <- c(chosen_arm, reward)

  # get a vector of results from chosen arm (arm, reward) and add to the results
  df_results_policy[i,] <- result_policy_i

  # TODO: combine to dataframe with all results
  df_results_at_t <- rbind(df_results_at_t, result_policy_i)

```

```

}

# save results in list
results <- list(df_results_of_policy = df_results_policy,
               df_sample_of_policy = df_during_policy)

return(results)
}

```

Task 3: simulate the epsilon greedy algorithm

Using the 'sim_greedy' function, run the epsilon greedy algorithm for the following parameters: $n_{\text{before simulation}} = 20$, $n_{\text{simulation}} = 200$, $\epsilon = 0.1$ and $\epsilon = 0.25$. Calculate the total reward based on the returned 'df_results_of_policy'.

```

# set parameters: observations before start simulation, then how many observations for simulation
n_before_sim <- 20
n_sim <- 200

# set the seed for reproducibility
set.seed(0)

# Use the sim_greedy function created in task 2 to simulate the results for epsilon = 0.1
result_sim_eps_01 <- sim_greedy(dfYahoo_for_sim,
                               n_before_sim=n_before_sim,
                               n_sim=n_sim,
                               epsilon=0.1,
                               interval=1)

# TODO: use the sim_greedy function created in task 2 to simulate the results for epsilon = 0.25
# assign the results to a variable called result_sim_eps_025
result_sim_eps_025 <- sim_greedy(dfYahoo_for_sim,
                                 n_before_sim=n_before_sim,
                                 n_sim=n_sim,
                                 epsilon=0.25,
                                 interval=1)

```

Task 4: plot performance for t

Compare the performance of the algorithm for $\epsilon = 0.1$ and $\epsilon = 0.25$ using a plot. The x-axis should show the steps t , and the y-axis the cumulative reward at point t . Which one of these performs better? Give a general explanation why we would expect this version of the algorithm to perform better?

```

# create a dataframe with the results per epsilon.
# calculate the cumulative sum over time
df_result_sims <- data.frame(t = 1:n_sim,
                             eps_01 = cumsum(result_sim_eps_01$df_results_of_policy$reward),
                             eps_025 = cumsum(result_sim_eps_025$df_results_of_policy$reward))

# melt the dataframe for ggplot visualization
df_result_sims_melted <- melt(df_result_sims, id = 't')

# TODO: Plot the reward in a line graph to compare performance for epsilon = 0.1 and epsilon = 0.25
epsilon_comparison <- ggplot(data=df_result_sims_melted, aes(x=t, y=value, col=variable)) +
  geom_line() +

```

```

theme_bw()+
xlab('Time') +
ylab('TotalReward') +
scale_color_manual(name= TeX('$\\epsilon$'),
                    values=c("darkblue", "red"),
                    labels=c('0.1', '0.25'))

```

Extra: if you have time left, consider implementing a grid-search, which is a method used to evaluate every possible combination of selected parameter values (e.g. for $n_{\text{before simulation}}$, interval, and ϵ) to determine the combination leading to the highest reward.

Calculating regret

Regret is the opportunity loss given what we could have received, and what we did receive. The optimal value, denoted by V^* , is the highest possible average reward we can achieve by selecting the best arm from the set of all arms. The total regret at time t is then:

$$L_t = E[\sum_{\tau=1}^t V^* - Q(a_\tau)] \quad (2)$$

Intuitively, it measures the difference between selecting the arm with the best average reward and the arms chosen by our algorithm.

Task 5: calculate regret for $\epsilon = 0.1$

Based on equation (2), calculate the regret for the greedy algorithm when $\epsilon = 0.1$.

```

# set the seed
set.seed(0)

# total reward for our algorithm
total_reward_eps_01 <- tail(df_result_sims$eps_01, n=1)

# obtain the average reward per arm
df_result_per_arm_sample <- result_sim_eps_01$df_sample_of_policy %>%
  group_by(arm) %>%
  summarise(avg_reward = mean(reward))

# TODO: calculate regret for the greedy algorithm when epsilon = 0.1 by using the following steps:
# 1. Select the average reward of the best arm
# 2. Determine the total reward of the best arm
# 3. Calculate the regret

# average reward of best arm
avg_reward_best_arm <- max(df_result_per_arm_sample$avg_reward)

# Total reward of the best arm
total_reward_best_arm <- round(n_sim * avg_reward_best_arm, 0)

# regret
regret <- total_reward_best_arm - total_reward_eps_01

```

Task 6: Brain teaser 1

Suppose that you are working for a company implementing this algorithm, but you come across a problem. The company has millions of customers, and your algorithm has to handle huge amounts of data. As a consequence, choosing an arm with your algorithm can take very long, and importantly, cost you money when outsourcing the computation online. So, you've come up with a solution. You would like to aggregate the data into batches, where you only recompute the optimal arm every k observations. This way, you can avoid some of the computational cost. However, you're worried this will hurt the performance of your algorithm. Thus, you decide to investigate how the algorithm performs over a range of k 's.

To calculate the performance of the algorithm for each k , assume the following: A successful arm pull gives a reward of 1. Plot total reward over a chosen range of k .

What you should do: set out some range of k 's that you would like to test over. Run the `sim_greedy()` function over these k . For each k , run the algorithm 100 times so you can get confidence intervals for the reward of each k policy. Finally, plot both total reward over all chosen k , and reason which k you think is best.

Answer: k is the `interval` argument of `sim_greedy()`. A small k recomputes the arm often and so carries the computational cost the company wants to avoid; a large k keeps a possibly wrong arm deployed for k observations, which could cost reward. Since reward is what we measure here, run the policy over a grid of k , repeat each k many times because a single run is noisy, and compare the mean total reward with its 95% confidence interval.

```
# grid of update intervals, and the number of runs to average over
k_values <- c(1, 2, 5, 10, 25, 50, 100, 200)
num_reps <- 100 # reward is noisy, so average over many runs (takes a couple of minutes)

set.seed(0)

df_k <- bind_rows(lapply(k_values, function(k){

  # total reward collected by the policy, for num_reps runs at this k
  runs <- replicate(num_reps, {

    res <- sim_greedy(dfYahoo_for_sim,
                      n_before_sim = n_before_sim,
                      n_sim = n_sim,
                      epsilon = 0.1,
                      interval = k)

    sum(res$df_results_of_policy$reward)
  })

  data.frame(k          = k,
             total_reward = mean(runs),
             se_reward   = sd(runs)/sqrt(num_reps))
}))

df_k

# the k with the highest average reward
best_k <- df_k$k[which.max(df_k$total_reward)]
best_k

# Plot: total reward over k, with 95% confidence intervals
ggplot(df_k, aes(x = k, y = total_reward)) +
  geom_errorbar(aes(ymin = total_reward - 1.96*se_reward, ymax = total_reward + 1.96*se_reward),
```

```

    width = 0.03, colour = 'grey60') +
  geom_line(colour = 'darkblue') +
  geom_point(colour = 'darkblue') +
  scale_x_log10(breaks = k_values) +
  labs(x = 'k (observations between updates)',
       y = 'Total reward',
       title = paste0('Total reward over k (mean of ', num_reps, ' runs)')) +
  theme_bw()

```

Reading the plot answers the question. Total reward turns out to be essentially flat in k on this data, k doesn't influence the success of the algorithm much: the confidence intervals of every k overlap, so we cannot say that updating every observation beats updating every 10th or every 100th.

A reasonable k is thus the largest one at which the algorithm still works well - about $k = 10$ here. Updating arm choice more often does not buy more accuracy, while every extra recalculation is computation the company pays for. Updating less often increases risk. Note that changing `n_sim`, the number of arms, or how far apart the arms' click rates lie can change the conclusion here.